| Course Name: | COMP 2233 Data Structures | | |
|---|---|---|---|
| Lecturers: | Assoc. Prof. Dr. Ömer ÇETİN<br>Research Assistant Giray KILIÇ<br>Research Asistant Hatice ÇAĞLAYANLAR | | |
| Lab. Number: | 3 | Lab. Date: | 06.10.2026 / 09.10.2026 |
| Objectives: | Exp. 1: C song array. Exp. 2: C++ contact vector and bubble sort. Analyze operation costs. | | |

| Course Learning Outcomes (CLO): | | Relevancy (1-5) |
|---|---|---|
| CLO (In1) | Students will be able to identify useful data structures to represent specific types of information. | 5 |
| CLO (Sk3) | Students will be able to design and implement linear data structures. | 5 |
| CLO (Co2) | Students will be able to design and implement efficient solutions for searching and sorting operations. | 3 |

## Pre-Lab Assignment

Prepare for both experiments.

**Experiment 1 Preparation (C)**

- **C Arrays: Review character arrays, capacity, indexing, and shifting.**
- **C Input: Review fgets, strtol, and null terminators.**

**Experiment 2 Preparation (C++): Review Contact, string, and vector.**

**C++ Vector and Sorting Preparation**

- **Menu: cin. Fields: getline; consume the menu line first. Bubble sort by surname, then name (O(N^2)).**
- **Prepare pseudocode and flowcharts for both menus.**

**Development Environment Setup**

- Compile with gcc -std=c11 and g++ -std=c++17.
- Test positions, empty lists, removal, sorting, and vector growth.

**Pre-Lab Self-Check Questions**

- *Check 1: Why shift songs left after removal?*
- *Check 2: What do vector size and capacity mean?*
- *Check 3: What fields are compared when sorting contacts?*

## Laboratory Experiment

**COMP 2233 / Data Structures Laboratory — Experiment 1 (C)**

**Topic: C One-Dimensional Array Operations for a Music Playlist**

**1. Problem Description**

Store playlist titles in a fixed one-dimensional C array. Add, remove, update, view, and swap songs.

**2. Constraints and Specifications**

- Capacity: 10 songs; track the active count.
- Title: Up to 63 characters. Positions start at 1.

**3. Experimental Tasks and Deliverables**

Complete the following steps:

1. Write pseudocode for playlist operations.
2. Draw a flowchart with position and capacity checks.
3. Array and Addition:
    - Use char songs[10][64], initialize three songs, and track count.
    - Append is O(1) when space is available.
4. Remove, Update, Swap, View:
    - Shift after removal; validate positions for update and swap. Display active songs.
    - Handle invalid positions, empty lists, and a full array.
5. Complexity:
    - Append, indexed update, and swap: O(1). Remove and view: O(N).

**Sample Terminal I/O (Experiment 1):**

```
First position: 1
Second position: 3
After swapping positions 1 and 3: Billie Jean, Take Five, Imagine.
Swap: O(1).
```

**COMP 2233 / Data Structures Laboratory — Experiment 2 (C++)**

**Topic: C++ Vector of Contact Structures and Bubble Sorting**

**1. Problem Description**

Store contacts in a C++ vector. Add, remove, update, view, and sort by surname, then name.

**2. Constraints and Specifications**

- **Vector starts empty and grows automatically.**
- **Data Fields per Contact:**
    - **Name and surname: string fields; no fixed length in the code.**
    - **Phone and e-mail: string fields; no fixed length in the code.**
    - **Birthday is prompted as DD/MM/YYYY; the code does not validate its format.**

**3. Experimental Tasks and Deliverables**

Complete the following steps:

1. **Write pseudocode for contact operations.**
2. **Draw a flowchart for the menu and position checks.**
3. **Struct and Vector Creation:**
    - Define Contact with five string fields and create vector<Contact>.

- Add with push_back; vector manages storage.
- Know size (contacts) and capacity (allocated slots).

4. **Contact Operations and Vector Lifetime:**
   - Use push_back to add, erase to remove, and indexing to update.
   - Display all fields. Bubble-sort by surname, then name.
   - Vector memory is released automatically at scope exit.
5. **Complexity:**
   - push_back: O(1) amortized. erase and view: O(N). Index access: O(1).
   - Bubble sort uses O(N^2) comparisons; string comparison also depends on text length.

**Sample Terminal I/O (Experiment 2):**

```
Name: Ada; Surname: Lovelace
Phone: 555-0100; E-mail: ada@example.com
Birthday: 10/12/1815
1. Ada Lovelace; 555-0100; ada@example.com; 10/12/1815
```

## Additional Knowledge (Supplementary Reference)

Key ideas:

1. **Contiguous Memory and O(1) Indexing**
   - **Array elements are contiguous; indexed access is O(1).**
   - **Array Address Calculation:**

Address(i) = Base Address + i * sizeof(Element)

C stores each title in a row of the two-dimensional character array.

2. **C++ Vector Capacity and Lifetime**
   - **A vector stores contacts contiguously; size counts contacts, capacity counts slots.**
   - The vector grows automatically when needed.
   - C++ releases vector storage at scope exit.
3. **Time and Space Complexity (Big-O)**
   - **Index access and array swap: O(1). String assignment depends on text length.**
     - **Append: O(1) for the array; O(1) amortized for vector push_back.**
     - **Removal and list display: O(N).**
   - **Storage: O(N) for both lists.**
4. **Contact Sorting**
   - **Sort is case-sensitive: surname first, then name.**
   - **Bubble sort swaps complete Contact records.**
   - **Bubble sort: O(N^2) comparisons.**
5. **Safe String Handling and Boundary Checks**
   - C text input uses fgets; C++ contact fields use getline.
   - Validate positions and reject additions to a full C array.
   - Test empty lists, invalid positions, swaps, sorting, and vector growth.

## Attachments

A. C template: lab3_experiment1_template.c;
B. C++ template: lab3_experiment2_template.cpp;

## References

1. Weiss, M. A. (2013). Data Structures and Algorithm Analysis in C (2nd ed.). Pearson.
2. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to Algorithms (4th ed.). MIT Press.
3. Kernighan, B. W., & Ritchie, D. M. (1988). The C Programming Language (2nd ed.). Prentice Hall.
4. ISO/IEC 5807:1985. Information processing — Documentation symbols and conventions for data, program and system flowcharts.